



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

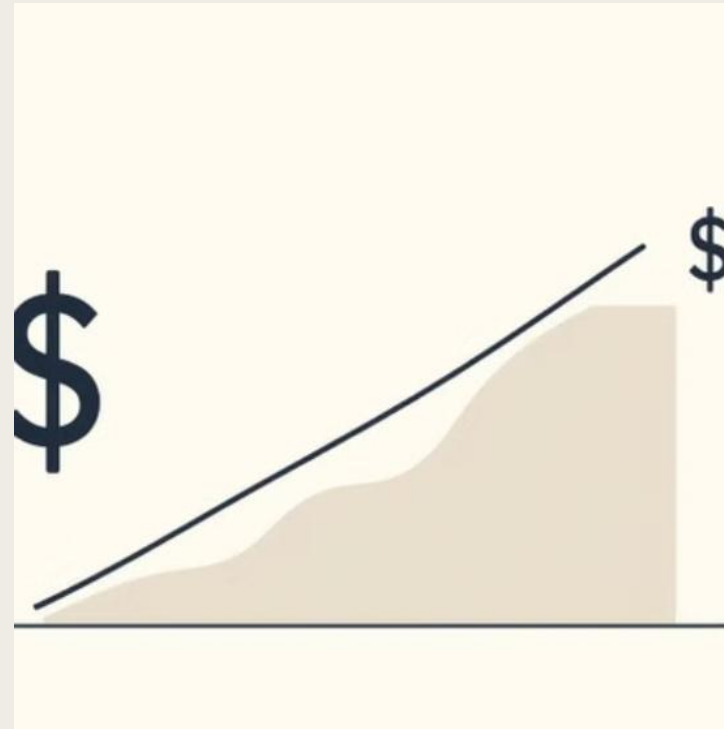
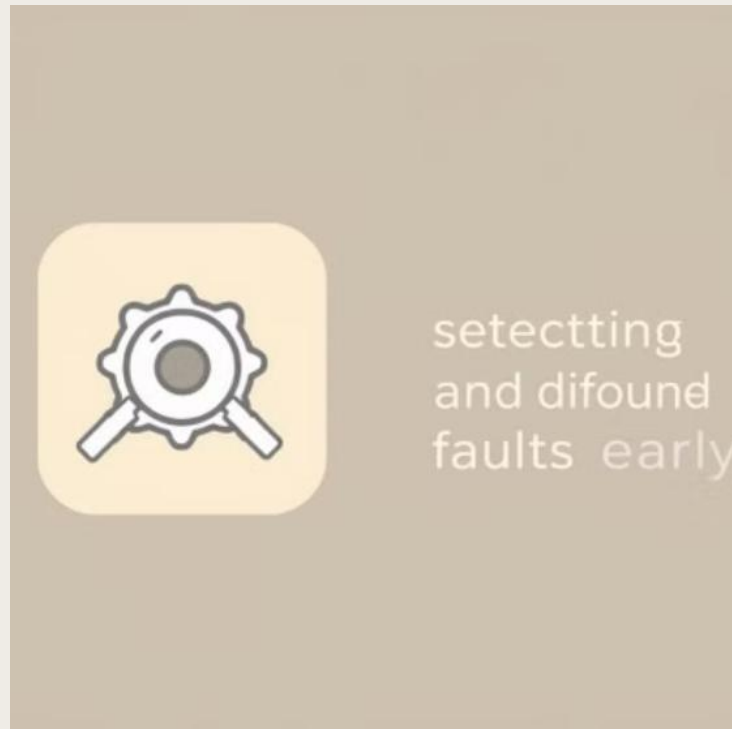
Testes de Sistemas

Professor: Alann Perini.

Este resumo apresenta os principais tópicos das aulas sobre Testes de Software.



Importância dos Testes de Software



Detectar e corrigir falhas antes do uso real é crucial.

Os testes reduzem os custos com a manutenção do sistema.

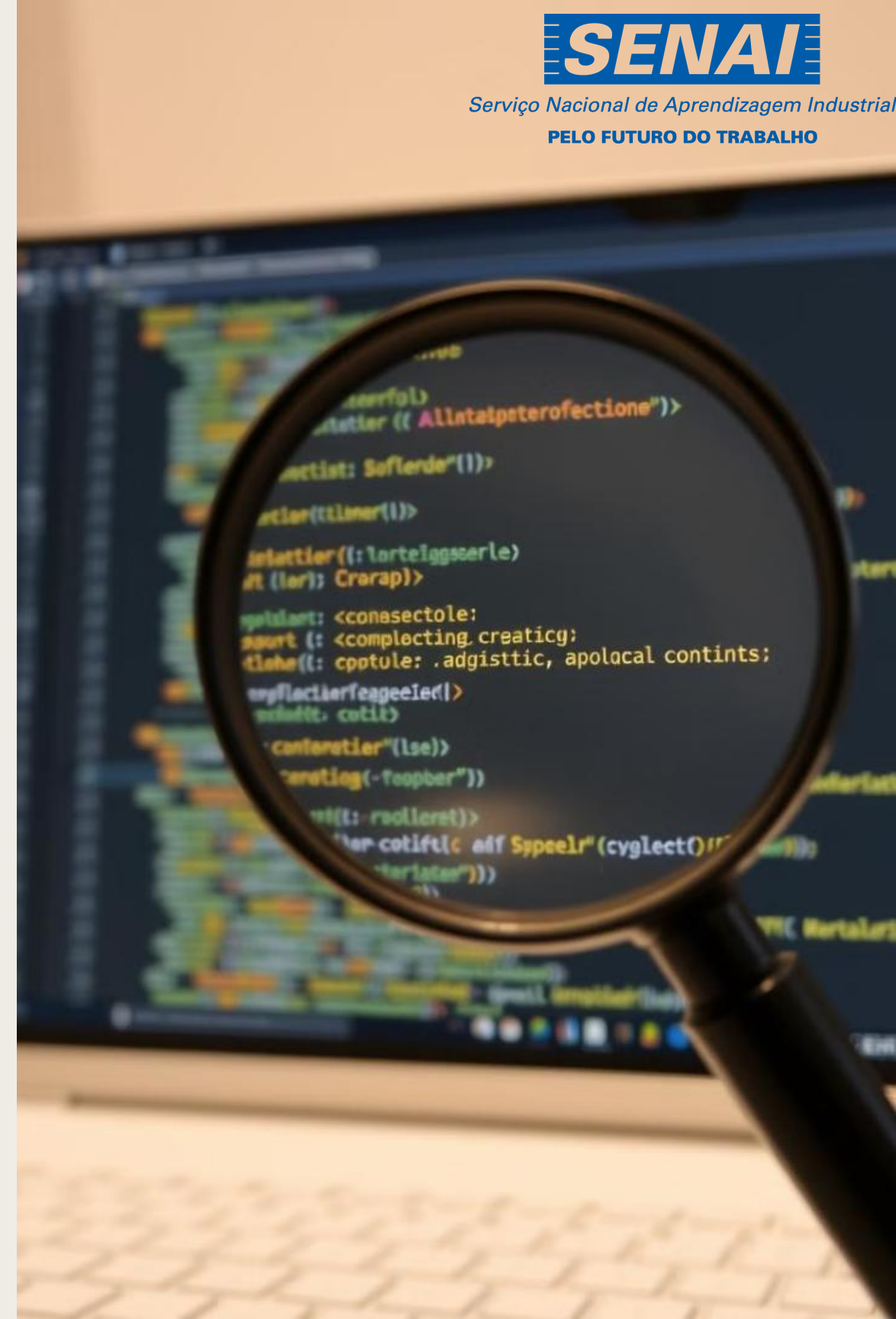
Testes aumentam a confiabilidade geral do software.

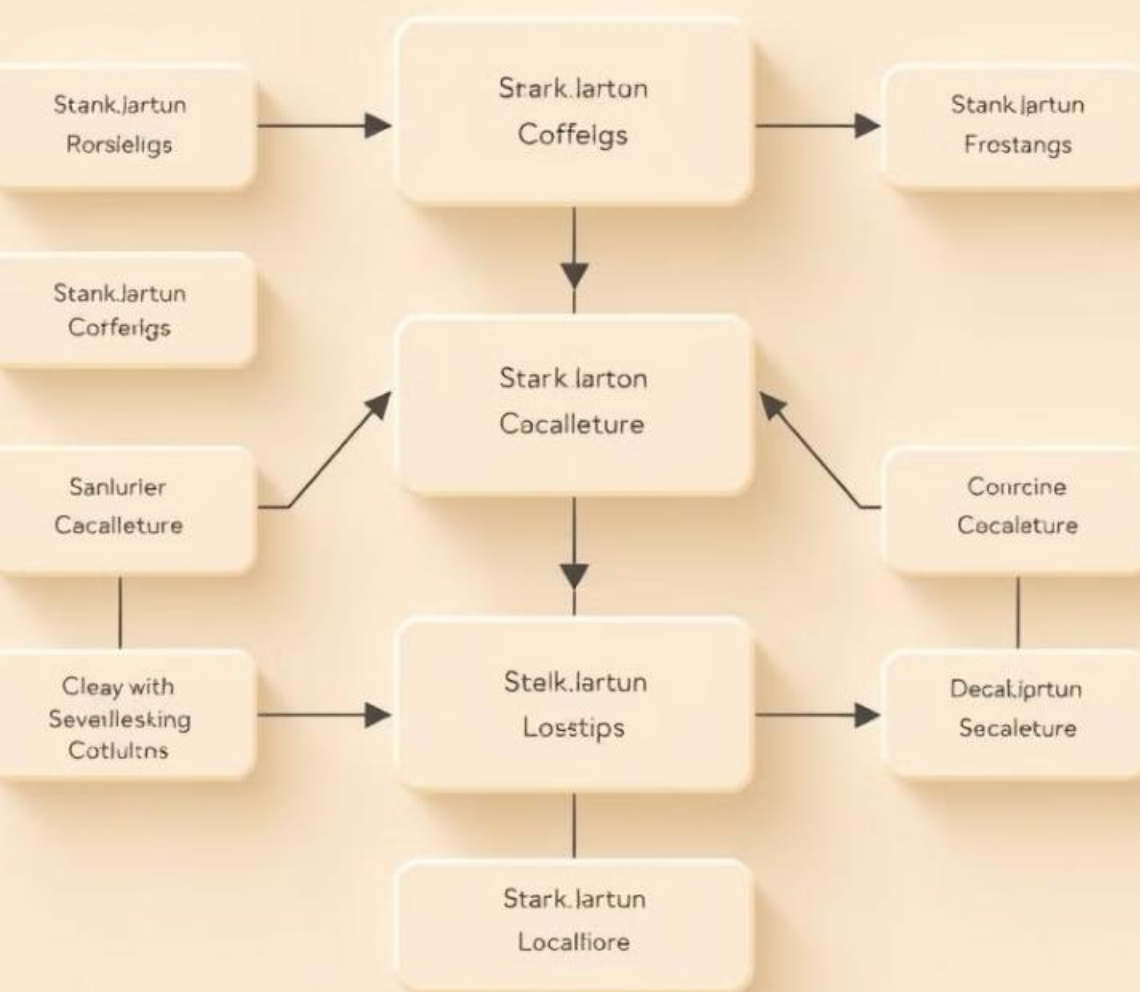
Testes de Unidade

Foco em componentes isolados do código.

Testam-se funções, métodos ou classes individuais.

Por exemplo, testar a função que calcula o imposto.





Testes de Integração

Verificam a interação entre módulos diferentes.

Um exemplo: a tela de cadastro integrada ao banco de dados.

São importantes para sistemas modulares.

Testes de Sistema



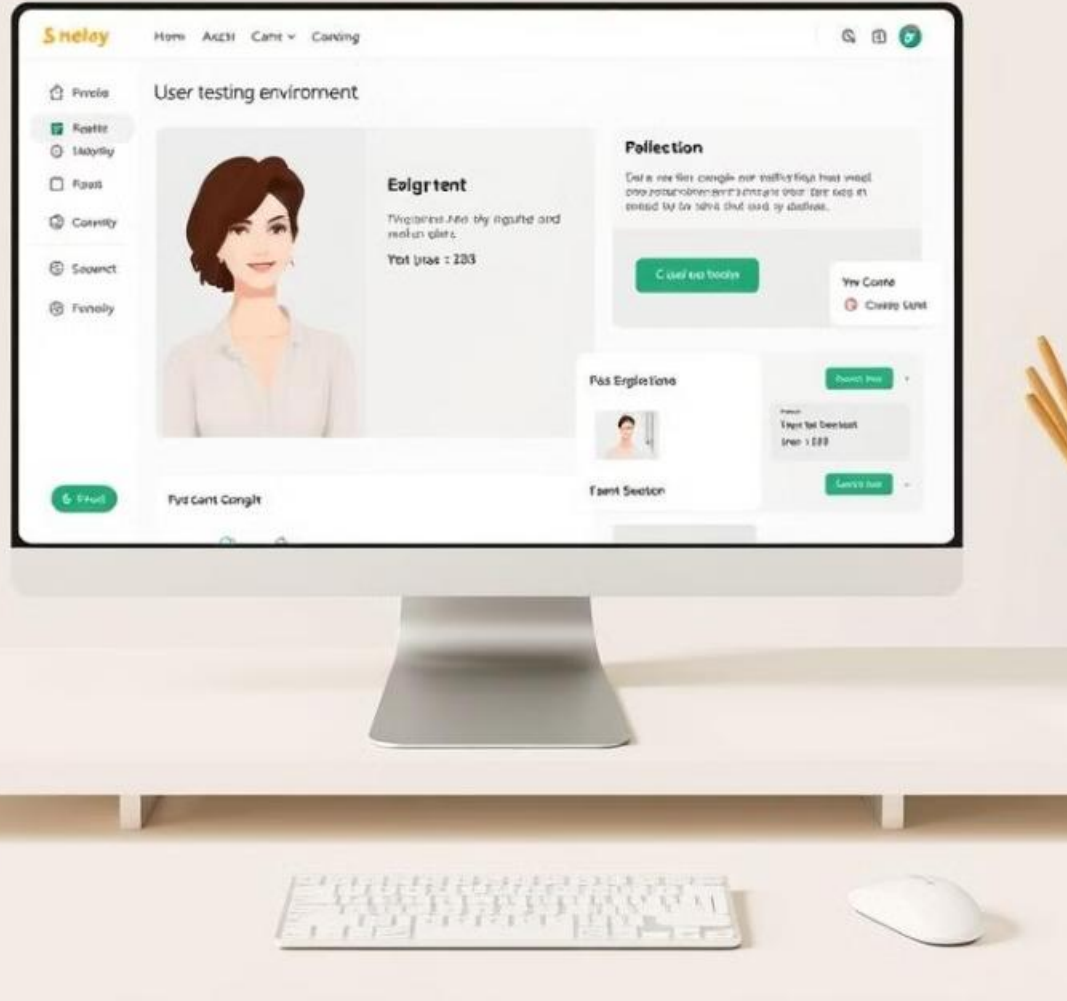
Avalia o sistema completo, simulando uso real pelo usuário.

Unidade: cálculo de frete.

Integração: autenticação + dashboard.

Sistema: checkout completo.

Aceitação: cliente testando emissão de nota.



Testes de Aceitação

Validação feita pelo cliente ou usuário final.

Baseado em critérios de aceitação definidos com antecedência pelo levantamento de requisitos.

Exemplo: cliente testa sistema de agendamento antes da homologação.

Testes de Regressão

Garantem que alterações no código não afetem o que já funciona.

Após atualizar o cadastro, garantir que o login ainda funcione.

São importantes para a manutenção contínua do sistema.

Regression Testing





Testes de Usabilidade

Avaliamos a experiência do usuário com a interface.

Examine a clareza dos botões e dos fluxos de navegação.

Importantes para interfaces amigáveis e acessíveis.



Testes de Segurança

Detectam falhas que comprometem dados ou acesso.

Teste de injeção SQL em formulário de login.

Crucial para sistemas bancários, governamentais e com dados sensíveis.

Caixa Branca, Preta e Cinza

Caixa Branca: analisa a lógica interna (ideal para devs).

Caixa Preta: testa sem ver o código.

Caixa Cinza: mistura os dois, bom para integração.

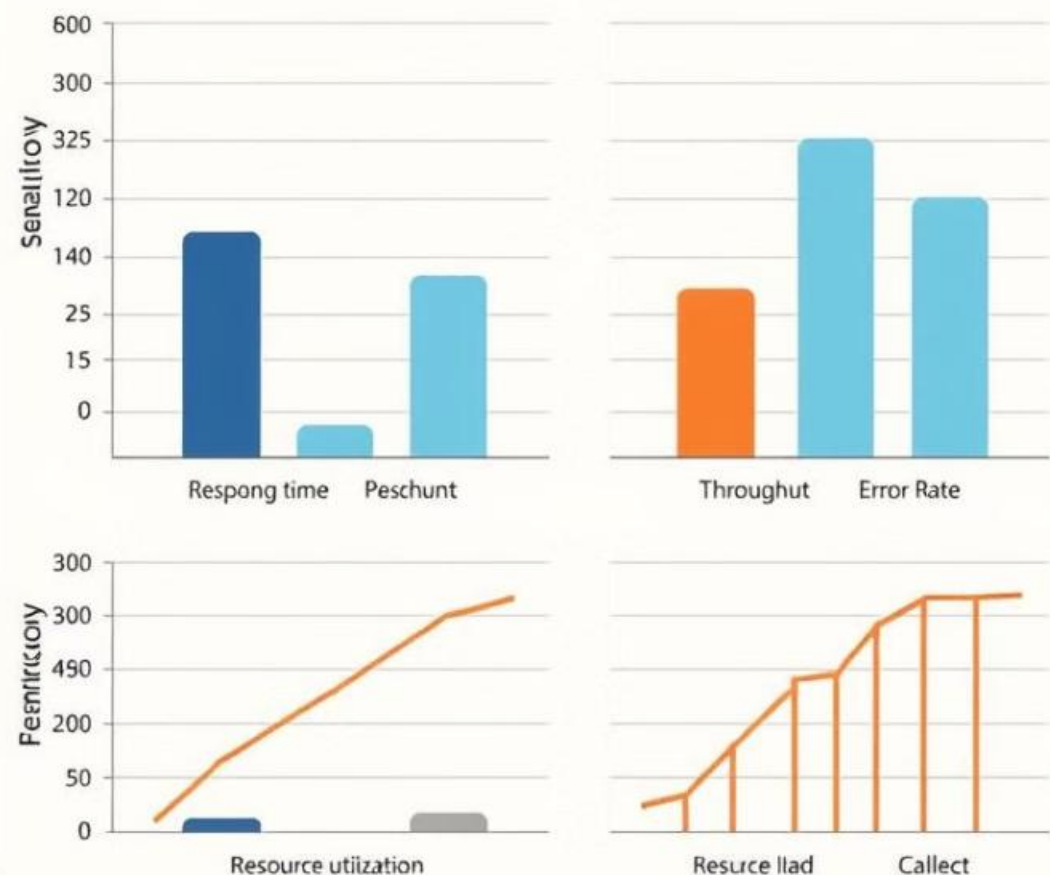
O Plano de Testes define o que, como e quem testa.

Controle via Trello, cronogramas e relatórios.

Acompanhamento de execuções e riscos.



System Performance Test



Testes de Performance

Verificam velocidade, tempo de resposta e uso de recursos.

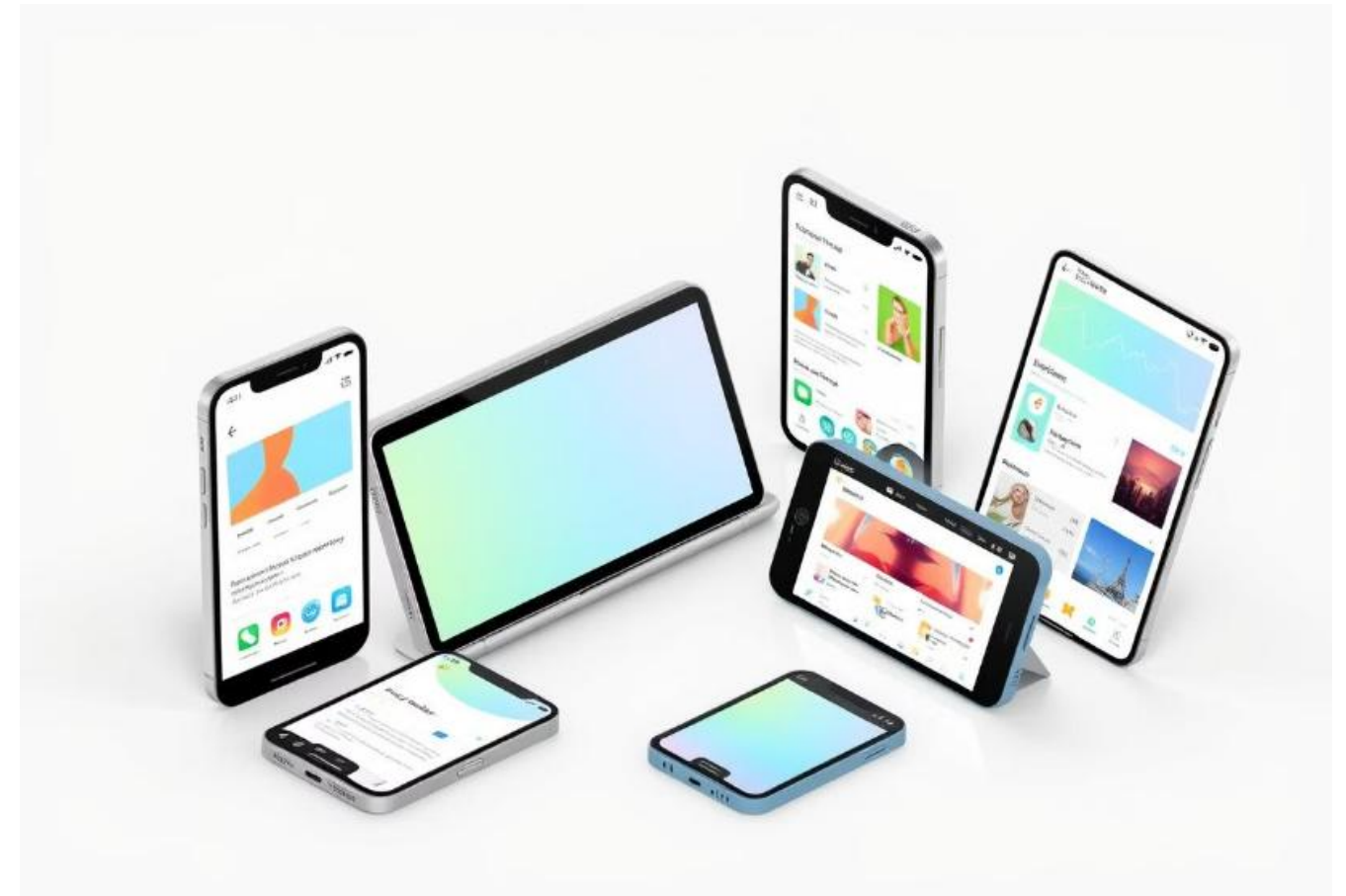
Ex: medir tempo de carregamento de página com 100 usuários. Ferramentas: JMeter, LoadRunner.

**System Performance
Under Load**

Teste de Compatibilidade

Verifica se o software funciona corretamente em diferentes ambientes. Isso inclui dispositivos, navegadores e sistemas operacionais.

- Testar em Android e iOS.
- Testar em diferentes navegadores.



Planejamento e Gerenciamento

1 Plano de Testes

Define o que, como e quem irá testar.
Documenta todo o processo.

2 Controle e Acompanhamento

Use Trello, cronogramas e relatórios para controlar.
Acompanhe as execuções e os riscos.

3 Gerenciamento de Riscos

Identifique e mitigue os riscos para garantir o sucesso.



Automação de Testes



Redução de esforço
Automatize para reduzir o esforço manual.



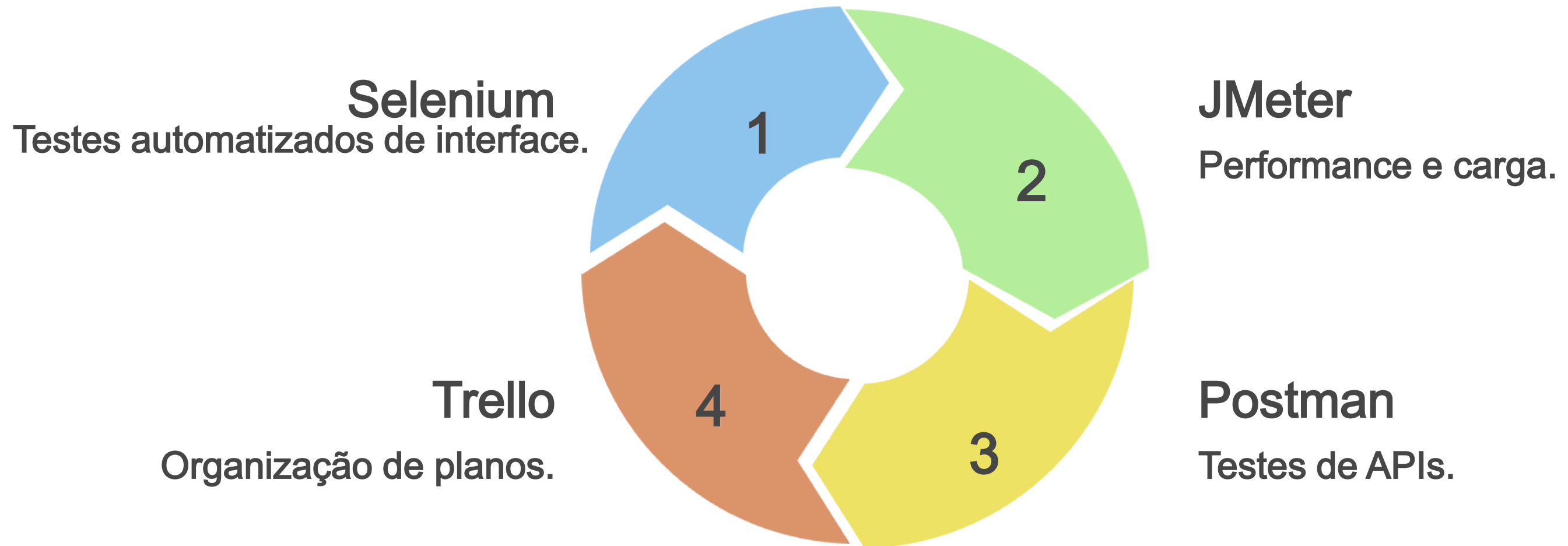
Aceleração
Acelera os testes repetitivos.



Regressão
Ideal para testes de regressão.

A automação de testes é crucial para aumentar a eficiência. Utilize Selenium para interface web. Use PyTest para funções Python.

Ferramentas Populares de Teste



Conclusão e Recomendações



- Combine diferentes tipos de teste conforme o contexto.
- Planeje, registre e automatize sempre que possível.
- Testes garantem qualidade e confiança no produto final.